Practical 8: Performance Optimization and Lazy Loading in React

CO/PO	CO1 / PO3, PO5
Objective	To improve frontend performance using lazy loading and code splitting techniques.
Prerequisites	• Practicals 1–6 completed multi-route React app with full-stack integration • Basic understanding of how the browser loads and parses JavaScript bundles
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 8 IBM: Developing Front-End Apps with React, Module 4 (advanced React features, hooks for optimization). Supplement with browser DevTools Performance/Network tab profiling.
Architecture / Diagram	

Diagram	`Before (single bundle):`
	`  main.bundle.js  ──────────►  [Home][Projects][Contact] all loaded upfront`
	
	`After (code-split):`
	`  main.bundle.js  ──► loads on first visit`
	`  Home.chunk.js  ──► loaded only when /  is visited`
	`  Projects.chunk.js ──► loaded only when /projects is visited`
	`  Contact.chunk.js ──► loaded only when /contact is visited`

Problem Definition	
Problem Statement	• Optimize the React frontend of the Task Management application by implementing lazy loading for route-based components. • Apply React.lazy() and Suspense to at least the Projects and Contact routes (or equivalent route-based components). • Provide a meaningful fallback UI while a lazy-loaded chunk is loading. • Measure and compare bundle size and load time before and after optimization using browser developer tools. • Document the before/after comparison with screenshots or recorded values.
Key Questions / Analysis	• What is the difference between the initial bundle and a lazy-loaded chunk in terms of when each is downloaded? • Why does lazy loading improve perceived performance even though the total amount of code downloaded eventually stays the same? • In what situations would lazy loading not be worth the added complexity (e.g., a very small app)?
Supplementary Problems	• Lazy load a heavy third-party component (e.g., a chart library) only when a specific page needs it. • Add a minimum-delay fallback (e.g., 300ms) to avoid a flickering loading state on fast connections. • Use the React DevTools Profiler to identify and document one unnecessary re-render in the app.
Key Skills Addressed	• Implementing route-based code splitting using React.lazy() and Suspense

	• Reading and interpreting the browser Network tab for chunk loading behavior • Measuring bundle size using build tooling output • Communicating a before/after performance comparison clearly
Learning Outcome	Students will be able to apply code splitting and lazy loading in React to reduce initial bundle size, and will be able to measure and explain the resulting performance impact.
Lab Session Step-by-Step	

Steps	1. Record baseline metrics before optimization build the app and check bundle size:
	`npm run build`
	`# note the output bundle size shown in the terminal`
	2. Open the Network tab in DevTools, reload the app, and note the total JS transferred and load time.
	3. Convert route imports to lazy imports:
	`import { lazy, Suspense } from 'react';`
	`const Projects = lazy(() => import('./pages/Projects'));`
	`const Contact = lazy(() => import('./pages/Contact'));`
	4. Wrap the Routes block with Suspense and a fallback:
	`<Suspense fallback={<div>Loading page...</div>}>`
	`  <Routes>`
	`  <Route path="/projects" element={<Projects />} />`
	`  <Route path="/contact" element={<Contact />} />`
	`  </Routes>`
	`</Suspense>`
	5. Rebuild the app and confirm separate chunk files are generated for each lazy route.
	6. Reload with Network tab open, throttle to "Slow 3G", and observe the fallback UI rendering before the chunk loads.
	7. Record post-optimization metrics and prepare the before/after comparison.

Tools / Technology / Resources Required	• Node.js (v18+), npm • React 18+ (lazy, Suspense) • Browser DevTools (Network + Performance tabs) • Vite build output
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours
Faculty Teaching Guide	
Guidance Notes	• Tell students in advance to capture baseline screenshots before making any changes this is required evidence for the rubric and cannot be reconstructed afterward. • Demonstrate Network throttling (Slow 3G) live so the fallback UI is actually visible on fast lab Wi-Fi the loading state may flash too quickly to notice otherwise. • Clarify that lazy loading delays when code downloads, not how much code downloads in total over a full session. • Walk through reading the Vite build output chunk list together once before students interpret their own.
Common Mistakes	• Not capturing baseline (before) metrics, making the comparison impossible to demonstrate. • Wrapping individual components in Suspense unnecessarily instead of wrapping at the route level. • Forgetting that React.lazy() only works with default exports, causing import errors. • Testing only on fast lab Wi-Fi without throttling, so the fallback UI is never actually observed.

Troubleshooting Guide

Symptom	Likely Cause	Fix
Element type is invalid error after adding lazy()	Lazy-loaded component is not a default export	Ensure the component file uses export default ComponentName
Fallback UI never appears	Chunk loads too fast on local network to observe	Enable Network throttling (Slow 3G) in DevTools to simulate a slower connection
No separate chunk files after build	Static import still used instead of dynamic import()	Confirm the route component uses lazy(() => import('./Component')) syntax, not a top-level import
Suspense fallback shows for the entire app, not just one route	Suspense wraps too broad a section of the component tree	Move Suspense to wrap only the Routes block, not the entire App component

Student Assignment / Post Laboratory Work

Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Lazy load one additional heavy component (e.g., a chart or large list) beyond the required routes. • Use React DevTools Profiler to identify and note one unnecessary re-render in the app.

GitHub Deliverables

Expected Repository Contents	• Same repository as Practical 6/7, updated with new commits • Lazy loading applied to at least 2 route-based components with Suspense fallback • Before/after screenshots or recorded metrics included in the README or a docs folder • At least one commit specifically for performance optimization

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
Conceptual understanding (2 Marks)	Explains what and why, not just what was done
Implementation (4 Marks)	Code written, runs, and is original
Correctness (2 Marks)	Output matches expected result
Reflection (1 marks)	Errors documented and explained
Lab file evidence (1 marks)	Completeness and presentation
Total: 10 marks	**Passing Threshold: 5 / 10 (50%)**